

Francisco Ortiz Tomás 2DAM

UT1. PROGRAMACIÓN MULTIPROCESO

ACTIVIDAD 4

Procesos en paralelo y documentación

Título: ¿Compensa repartir el trabajo entre varios procesos?

Descripción:

Repartir una tarea grande entre varios procesos suele acelerarla, pero no siempre tanto como se espera. En esta actividad ejecutas la misma carga de trabajo de dos formas, una detrás de otra y todas a la vez, mides los tiempos y documentas el resultado. Es la comprobación que hace cualquier desarrollador antes de afirmar que su solución es más rápida.

Objetivos:

- Desarrollar una aplicación que reparta una tarea entre varios procesos en paralelo.
- Medir y comparar el rendimiento obtenido frente a la ejecución secuencial.
- Documentar con Javadoc y depurar la aplicación desarrollada.

Instrucciones:

Cada subproceso escribe en su propio fichero. Si los cuatro escribieran en el mismo, se pisarían unos a otros y el resultado no sería fiable. Genera solo letras de la A a la Z en mayúscula, sin tildes ni eñes.

1. Crea la clase **Lenguaje**. Recibe como argumento el nombre de su fichero de salida y escribe en él 100.000 líneas de 60 letras aleatorias cada una. Termina con **System.exit(0)**. El fichero se crea en la carpeta del proyecto: basta con usar el nombre que recibe, sin ninguna ruta delante.
2. Crea la clase **Colaborar** con un método que ejecute cuatro procesos **Lenguaje** de forma secuencial: lanza uno, espera con **waitFor()** y solo entonces lanza el siguiente. Cada proceso escribe en su propio fichero: **seq1.txt**, **seq2.txt**, **seq3.txt** y **seq4.txt**. Mide el tiempo total con **System.nanoTime()**.
3. Añade un segundo método que los ejecute en paralelo: lanza los cuatro seguidos guardando cada **Process** en una lista y recorre después la lista llamando a **waitFor()**. Aquí los ficheros se llaman **par1.txt**, **par2.txt**, **par3.txt** y **par4.txt**. Mide el tiempo igual que antes.

Si esperas a cada proceso justo después de lanzarlo, no hay paralelismo.

4. Comprueba que se han creado los ocho ficheros, que cada uno tiene 100.000 líneas y que ningún **waitFor()** ha devuelto un valor distinto de 0. Completa

esta tabla y responde en cinco líneas por qué la mejora no llega a ser igual al número de núcleos:

Secuencial (ms)	Paralelo (ms)	Mejora	Núcleos disponibles

5. **Documenta ambas clases con Javadoc:** descripción de la clase, **@author** con tu nombre y **@param** en los métodos. Genera después la documentación desde IntelliJ con Tools → Generate JavaDoc.

Rúbrica:

0	No se entrega, o lo entregado no corresponde a la actividad.
3	La clase Lenguaje genera su fichero, pero Colaborar no llega a lanzar los cuatro subprocesos.
5	Colaborar ejecuta los cuatro subprocesos y se generan los ficheros, pero solo implementa uno de los dos modos o no mide tiempos.
7	Implementa los dos modos y aporta los dos tiempos, pero en el modo paralelo espera a cada proceso nada más lanzarlo, o no verifica los ficheros ni los códigos de salida.
9	Los dos modos son correctos (en el paralelo se lanza primero y se espera después), la tabla del apartado 4 está completa y verificada, la respuesta explica al menos dos causas de la pérdida de rendimiento y el Javadoc está generado con el autor visible.
10	Además, amplía la medición por iniciativa propia (otro número de procesos u otra carga de trabajo) y razona el resultado con datos, o aporta una captura del depurador que evidencie los cuatro procesos ya lanzados.

Entrega:

- **Código fuente de Lenguaje y Colaborar pegado como texto.** No se admiten capturas de pantalla del código.

LENGUAJE

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Random;

/**
 * Genera un fichero con 100.000 líneas de 60 letras
 * aleatorias en mayúscula de la A a la Z.
 *
 * Cada ejecución de esta clase corresponde a un proceso
```



```
* independiente lanzado desde la clase Colaborar.
*
* @author Francisco Ortiz
* @version 1.0
* @since 29/09/2026
*/
public class Lenguaje {

    // Número de líneas que tendrá cada fichero
    private static final int NUM_LINEAS = 100000;

    // Número de letras que tendrá cada línea
    private static final int LETRAS_LINEA = 60;

    /**
     * Método principal.
     *
     * Recibe como argumento el nombre del fichero
     * que debe generar.
     *
     * @param args contiene el nombre del fichero de salida
     */
    public static void main(String[] args) {

        // Comprobamos que se haya recibido
        // el nombre del fichero
        if (args.length == 0) {
            System.out.println("No se ha indicado el nombre del fichero.");

            // Terminamos indicando que ha ocurrido un error
            System.exit(1);
        }
        // Guardamos el nombre del fichero recibido
        String nombreFichero = args[0];

        // Creamos el generador de números aleatorios
        Random random = new Random();

        try {
            // Abrimos el fichero para escribir
            FileWriter escritor = new FileWriter(nombreFichero);

            // Generamos las 100.000 líneas
            for (int i = 0; i < NUM_LINEAS; i++) {

                // Generamos las 60 letras de cada línea
                for (int j = 0; j < LETRAS_LINEA; j++) {

                    /*
                     * Generamos un número entre 0 y 25
                     * y lo sumamos al código de la letra 'A'.
                     *
                     * De esta forma obtenemos una letra
                     * aleatoria entre A y Z.
                     */
                    char letra = (char) ('A' + random.nextInt(26));
```



```
        // Escribimos la letra en el fichero
        escritor.write(letra);
    }

    // Después de las 60 letras,
    // hacemos un salto de línea
    escritor.write("\n");
}

// Cerramos el fichero
escritor.close();

// El proceso termina correctamente
System.exit(0);

} catch (IOException e) {

    // Mostramos el error si no se puede escribir
    System.out.println("Error al escribir el fichero: " + e.getMessage());

    // Indicamos que el proceso ha terminado con error
    System.exit(1);
}
}
```

COLABORAR

- ```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

/**
 * Lanza varios procesos Lenguaje de forma secuencial y en paralelo
 * para comparar los tiempos de ejecución.
 * <p>
 * Además, realiza las pruebas con 2, 4 y 8 procesos para comprobar
 * cómo afecta el número de procesos al rendimiento.
 *
 * @author Francisco Ortiz
 * @version 2.0
 * @since 29/09/2026
 */
public class Colaborar {

 // Número de líneas que esperamos encontrar en cada fichero
 private static final int LINEAS_ESPERADAS = 100000;

 /**
 * AMPLIACIÓN EXTRA:
 * Además de la prueba obligatoria con 4 procesos, realizamos
 * pruebas adicionales con 2 y 8 procesos.
```



```
*
* El objetivo es comparar cómo cambia el rendimiento al aumentar
* o disminuir el número de procesos y poder razonar los resultados
* obtenidos con datos reales.
*/
private static final int[] PRUEBAS_PROCESOS = {2, 4, 8};

/**
 * Método principal del programa.
 * Realiza las pruebas con 2, 4 y 8 procesos,
 * tanto de forma secuencial como paralela.
 *
 * @param args argumentos del programa
 */
public static void main(String[] args) {

 try {

 // Obtenemos el número de procesadores disponibles en el equipo
 int nucleos = Runtime.getRuntime().availableProcessors();

 System.out.println("Núcleos disponibles: " + nucleos);

 System.out.println("\n=====");
 System.out.println(" COMPARACIÓN DE RENDIMIENTO");
 System.out.println("=====");

 // Recorremos las diferentes pruebas: 2, 4 y 8 procesos
 for (int numProcesos : PRUEBAS_PROCESOS) {

 System.out.println("\n-----");
 System.out.println("PRUEBA CON " + numProcesos + " PROCESOS");
 System.out.println("-----");

 // Ejecutamos los procesos de forma secuencial
 // y guardamos el tiempo total
 long tiempoSecuencial = ejecutarSecuencial(numProcesos);

 // Ejecutamos los procesos de forma paralela
 // y guardamos el tiempo total
 long tiempoParalelo = ejecutarParalelo(numProcesos);

 // Calculamos la mejora obtenida con la ejecución paralela
 double mejora =
 (double) tiempoSecuencial / tiempoParalelo;

 // Mostramos los resultados de esta prueba
 System.out.println("\n---- RESULTADOS ----");

 System.out.println("Secuencial: " + tiempoSecuencial + " ms");

 System.out.println("Paralelo: " + tiempoParalelo + " ms");

 System.out.printf("Mejora: %.2f veces\n", mejora);

 // Comprobamos que los ficheros secuenciales
```



```
// se hayan generado correctamente
System.out.println("\n----- FICHEROS SECUENCIALES -----");

comprobarFicheros("seq", numProcesos);

// Comprobamos que los ficheros paralelos
// se hayan generado correctamente
System.out.println("\n----- FICHEROS PARALELOS -----");

comprobarFicheros("par", numProcesos);
}

} catch (IOException e) {

 // Error relacionado con los ficheros o procesos
 System.out.println("Error: " + e.getMessage());

} catch (InterruptedException e) {

 // Error si alguno de los procesos es interrumpido
 System.out.println("Se ha interrumpido un proceso: " + e.getMessage());
}
}

/**
 * Lanza un proceso Lenguaje indicando el fichero
 * donde debe escribir.
 *
 * @param nombreFichero nombre del fichero de salida
 * @return proceso que se ha lanzado
 * @throws IOException si ocurre un error al iniciar el proceso
 */
private static Process lanzar(String nombreFichero)
 throws IOException {

 // Obtenemos la ruta del ejecutable de Java
 String java = System.getProperty("java.home") + "\\bin\\java.exe";
 /**
 * Creamos el ProcessBuilder indicando:
 *
 * 1. El ejecutable de Java.
 * 2. El classpath de la aplicación.
 * 3. La clase Lenguaje que queremos ejecutar.
 * 4. El nombre del fichero que debe crear.
 */
 ProcessBuilder pb = new ProcessBuilder(java, "-cp", System.getProperty("java.class.path"),
 Lenguaje.class.getName(), nombreFichero);

 // Iniciamos el proceso y lo devolvemos
 return pb.start();
}

/**
 * Ejecuta una cantidad determinada de procesos
 * de forma secuencial.

```



```
* <p>
* Cada proceso debe terminar antes de lanzar
* el siguiente.
*
* @param numProcesos número de procesos que queremos ejecutar
* @return tiempo total en milisegundos
* @throws InterruptedException si se interrumpe un proceso
* @throws IOException si ocurre un error al lanzar un proceso
*/
private static long ejecutarSecuencial(int numProcesos)
 throws InterruptedException, IOException {

 // Guardamos el momento de inicio
 long inicio = System.nanoTime();

 // Lanzamos los procesos uno detrás de otro
 for (int i = 1; i <= numProcesos; i++) {

 /*
 * Incluimos el número total de procesos en el nombre
 * para no mezclar los ficheros de las distintas pruebas.
 *
 * Ejemplo:
 * seq2_1.txt
 * seq4_1.txt
 * seq8_1.txt
 */
 String nombreFichero = "seq" + numProcesos + "_" + i + ".txt";

 // Lanzamos el proceso
 Process proceso = lanzar(nombreFichero);

 /*
 * Esperamos a que termine antes de continuar.
 * Esto hace que la ejecución sea SECUENCIAL.
 */
 int codigo = proceso.waitFor();

 // Comprobamos que haya terminado correctamente
 if (codigo != 0) {

 System.out.println("El proceso " + i + " ha fallado (código " + codigo + ")");
 }
 }

 // Calculamos el tiempo total y lo pasamos
 // de nanosegundos a milisegundos
 return (System.nanoTime() - inicio) / 1_000_000;
}

/**
* Ejecuta una cantidad determinada de procesos
* de forma paralela.
* <p>
* Primero se lanzan todos los procesos y después
```



```

 * se espera a que terminen.
 *
 * @param numProcesos número de procesos que queremos ejecutar
 * @return tiempo total en milisegundos
 * @throws InterruptedException si se interrumpe un proceso
 * @throws IOException si ocurre un error al lanzar un proceso
 */
 private static long ejecutarParalelo(int numProcesos)
 throws InterruptedException, IOException {

 // Guardamos el momento de inicio
 long inicio = System.nanoTime();

 // Lista donde guardaremos todos los procesos lanzados
 List<Process> procesos = new ArrayList<>();

 /*
 * PRIMER BUCLE: Lanzamos todos los procesos sin esperar
 * a que termine ninguno.
 */
 for (int i = 1; i <= numProcesos; i++) {

 String nombreFichero = "par" + numProcesos + "_" + i + ".txt";

 // Lanzamos el proceso
 Process proceso = lanzar(nombreFichero);

 // Guardamos el proceso en la lista
 procesos.add(proceso);
 }

 /*
 * En este punto todos los procesos ya han sido lanzados.
 *
 * Por ejemplo, en la prueba de 8 procesos,
 * los 8 procesos están ejecutándose simultáneamente
 * siempre que el sistema disponga de recursos.
 */

 /*
 * SEGUNDO BUCLE: Ahora esperamos a que todos los procesos terminen.
 */
 for (Process proceso : procesos) {

 int codigo = proceso.waitFor();

 // Comprobamos que haya terminado correctamente
 if (codigo != 0) {
 System.out.println("Un proceso paralelo ha fallado " + "(código " + codigo + ")");
 }
 }

 // Calculamos el tiempo total y lo pasamos
 // de nanosegundos a milisegundos
 return (System.nanoTime() - inicio) / 1_000_000;
 }
}
```



```
/**
 * Comprueba los ficheros generados en una prueba
 * y verifica que tengan exactamente 100.000 líneas.
 *
 * @param tipo "seq" para secuencial o "par" para paralelo
 * @param numProcesos número de procesos de la prueba
 * @throws IOException si ocurre un error al leer un fichero
 */
private static void comprobarFicheros(
 String tipo, int numProcesos) throws IOException {

 // Recorremos todos los ficheros de esta prueba
 for (int i = 1; i <= numProcesos; i++) {

 // Construimos el nombre del fichero
 String nombreFichero = tipo + numProcesos + "_" + i + ".txt";

 // Contamos las líneas que contiene
 int lineas = contarLineas(nombreFichero);

 // Comprobamos que tenga exactamente 100.000 líneas
 if (lineas == LINEAS_ESPERADAS) {
 System.out.println(nombreFichero + ": " + lineas + " líneas - CORRECTO");
 } else {
 System.out.println(nombreFichero + ": " + lineas + " líneas - ERROR");
 }
 }
}

/**
 * Cuenta el número de líneas que contiene un fichero.
 *
 * @param nombreFichero nombre del fichero que queremos comprobar
 * @return número de líneas encontradas
 * @throws IOException si ocurre un error al leer el fichero
 */
private static int contarLineas(String nombreFichero)
 throws IOException {

 // Abrimos el fichero para leerlo línea por línea
 BufferedReader lector = new BufferedReader(new FileReader(nombreFichero));

 // Variable donde guardamos el número de líneas
 int contador = 0;

 // Leemos hasta llegar al final del fichero
 while (lector.readLine() != null) {
 contador++;
 }
 lector.close();

 // Devolvemos el número total de líneas
 return contador;
}
```

- **Captura de la consola con los dos tiempos medidos y la tabla del apartado 4 completa.**

The screenshot shows a Java application window titled 'Performance' with a 'Show Results' button. The console output is as follows:

```

C:\Users\DAN2\jdk\remurin-25.0.4.1\bin\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2020.2.2\lib\idea
Núcleos disponibles: 12

=====
COMPARACIÓN DE RENDIMIENTO
=====

PRUEBA CON 2 PROCESOS

----- RESULTADOS -----
Secuencial: 1025 ms
Paralelo: 349 ms
Mejora: 2,94 veces

```

Overlaid on the console is a Windows PowerShell window with the following text:

```

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

PS C:\Users\DAN2> Francisco Ortiz Tomas

```

| Secuencial (ms) | Paralelo (ms) | Mejora | Núcleos disponibles |
|-----------------|---------------|--------|---------------------|
| 1025            | 349           | 2,94   | 12                  |

- **Captura del index.html de la documentación generada, con tu nombre visible en el @author.**



## Class Lenguaje

java.lang.Object<sup>Ⓔ</sup>  
Lenguaje

```
public class Lenguaje
extends ObjectⒺ
```

Genera un fichero con 100.000 líneas de 60 letras aleatorias en mayúscula de la A a la Z. Cada ejecución de esta clase corresponde a un proceso independiente lanzado desde la clase Colaborar.

### Since:

29/09/2026

### Version:

1.0

### Author:

Francisco Ortiz

## Constructor Summary

### Constructors

| Constructor | Description |
|-------------|-------------|
| Lenguaje()  |             |

## Method Summary

### All Methods Static Methods Concrete Methods

| Modifier and Type | Method                            | Description       |
|-------------------|-----------------------------------|-------------------|
| static void       | main(String <sup>Ⓔ</sup> [] args) | Método principal. |

### Methods inherited from class Object<sup>Ⓔ</sup>

clone<sup>Ⓔ</sup>, equals<sup>Ⓔ</sup>, finalize<sup>Ⓔ</sup>, getClass<sup>Ⓔ</sup>, hashCode<sup>Ⓔ</sup>, notify<sup>Ⓔ</sup>, notifyAll<sup>Ⓔ</sup>, toString<sup>Ⓔ</sup>, wait<sup>Ⓔ</sup>, wait<sup>Ⓔ</sup>, wait<sup>Ⓔ</sup>

## Constructor Details



Unnamed Package > Colaborar

Search documentation (type /)

## Class Colaborar

java.lang.Object<sup>↗</sup>  
Colaborar

---

```
public class Colaborar
extends Object↗
```

Lanza varios procesos Lenguaje de forma secuencial y en paralelo para comparar los tiempos de ejecución.

Además, realiza las pruebas con 2, 4 y 8 procesos para comprobar cómo afecta el número de procesos al rendimiento.

**Since:**  
29/09/2026

**Version:**  
2.0

**Author:**  
Francisco Ortiz

### Constructor Summary

| Constructors |             |
|--------------|-------------|
| Constructor  | Description |
| Colaborar()  |             |

### Method Summary

| All Methods       |                                   |                                |
|-------------------|-----------------------------------|--------------------------------|
| Static Methods    |                                   |                                |
| Concrete Methods  |                                   |                                |
| Modifier and Type | Method                            | Description                    |
| static void       | main(String <sup>↗</sup> [] args) | Método principal del programa. |

Methods inherited from class Object<sup>↗</sup>

clone<sup>↗</sup>, equals<sup>↗</sup>, finalize<sup>↗</sup>, getClass<sup>↗</sup>, hashCode<sup>↗</sup>, notify<sup>↗</sup>, notifyAll<sup>↗</sup>, toString<sup>↗</sup>, wait<sup>↗</sup>, wait<sup>↗</sup>, wait<sup>↗</sup>

### Constructor Details